

Guía de Estudio: Manejo de Fechas y Horas con `datetime` en Python

Autor: Manus AI

Introducción al Módulo `datetime`

El módulo `datetime` de Python proporciona clases para trabajar con fechas y horas de manera sencilla y eficiente. Es fundamental para cualquier aplicación que necesite manipular información temporal. Las clases principales son:

- `date` : Para trabajar solo con fechas (año, mes, día).
- `time` : Para trabajar solo con horas (hora, minuto, segundo, microsegundo).
- `datetime` : La combinación de `date` y `time` (año, mes, día, hora, minuto, segundo, microsegundo).
- `timedelta` : Para representar una duración, la diferencia entre dos fechas o tiempos.

Patrones Clave para Exámenes

1. Obtener la Fecha y Hora Actual

Para obtener la fecha y hora actual, se utiliza el método `now()` de la clase `datetime`. Es crucial para registrar eventos o establecer puntos de referencia temporales.

```
from datetime import datetime

ahora = datetime.now()
print(f"Fecha y hora actual: {ahora}")
# Ejemplo de salida: 2026-05-22 10:30:45.123456
```

2. Crear Objetos `datetime` Específicos

Se pueden crear objetos `datetime` para fechas y horas específicas, lo cual es útil para pruebas o para trabajar con datos predefinidos.

```
from datetime import datetime

fecha_especifica = datetime(2026, 12, 25, 10, 0, 0) # Año, mes, día, hora,
minuto, segundo
print(f"Fecha específica: {fecha_especifica}")
# Ejemplo de salida: 2026-12-25 10:00:00
```

3. Formatear Fechas y Horas (`strftime`)

`strftime` (string format time) permite convertir un objeto `datetime` en una cadena de texto con un formato específico. Es esencial para la presentación de fechas a los usuarios o para la interoperabilidad con otros sistemas.

Código de Formato	Descripción	Ejemplo para 2026-05-22 17:47:00
%Y	Año con siglo	2026
%m	Mes como número	05
%d	Día del mes	22
%H	Hora (24 horas)	17
%M	Minuto	47
%S	Segundo	00
%A	Día de la semana	Friday
%B	Nombre completo del mes	May
%a	Día de la semana abreviado	Fri
%b	Mes abreviado	May

```
from datetime import datetime

ahora = datetime.now()
formato_ejemplo = ahora.strftime("%d del %m de %Y a las %H:%M")
print(f"Fecha formateada: {formato_ejemplo}")
# Ejemplo de salida: 22 del 05 de 2026 a las 17:47
```

4. Parsear Cadenas a Objetos `datetime` (`strptime`)

`strptime` (string parse time) es el inverso de `strftime`. Permite convertir una cadena de texto que representa una fecha/hora en un objeto `datetime`. Es fundamental para procesar entradas de usuario o datos de archivos.

```
from datetime import datetime

cadena_fecha = "14/06/2026"
formato_cadena = "%d/%m/%Y"

try:
    fecha_parseada = datetime.strptime(cadena_fecha, formato_cadena)
    print(f"Fecha parseada: {fecha_parseada}")
    # Ejemplo de salida: 2026-06-14 00:00:00
except ValueError:
    print("La cadena no coincide con el formato esperado.")
```

Patrón clave: El formato de `strptime` debe coincidir *exactamente* con el formato de la cadena de entrada. Cualquier discrepancia (espacios extra, separadores incorrectos, etc.) resultará en un `ValueError`.

5. Realizar Operaciones con Fechas (`timedelta`)

La clase `timedelta` se utiliza para representar una duración y permite sumar o restar tiempo a un objeto `datetime`. Esto es esencial para calcular fechas futuras o pasadas.

```
from datetime import datetime, timedelta

ahora = datetime.now()
intervalo = timedelta(days=200) # Un intervalo de 200 días

fecha_futura = ahora + intervalo
print(f"Fecha actual: {ahora.strftime('%Y-%m-%d')}")
print(f"Fecha en 200 días: {fecha_futura.strftime('%Y-%m-%d')}")
```

6. Comparar Objetos `datetime`

Los objetos `datetime` se pueden comparar directamente usando operadores relacionales (`<`, `>`, `<=`, `>=`, `==`, `!=`). Esto es útil para determinar si una fecha es anterior o posterior a otra.

```
from datetime import datetime

fecha1 = datetime(2026, 1, 1)
fecha2 = datetime(2026, 1, 15)
fecha3 = datetime(2026, 1, 1)

print(f"Fecha1 < Fecha2: {fecha1 < fecha2}") # True
print(f"Fecha1 == Fecha3: {fecha1 == fecha3}") # True
```

7. Manejo de Excepciones al Parsear Fechas

Como se mencionó en `strptime`, es común que las cadenas de fecha no tengan el formato esperado. Utilizar bloques `try-except` con `ValueError` es una buena práctica para manejar estas situaciones de forma robusta.

```

from datetime import datetime

cadenas_fechas = ["13/02/25", "hola carmela", "14/06/2026"]
formato = "%d/%m/%y" # Para "13/02/25"
formato_largo = "%d/%m/%Y" # Para "14/06/2026"

fechas_correctas = []
fechas_incorrectas = []

for cadena in cadenas_fechas:
    es_correcta = False
    try:
        # Intentar con el formato corto
        fecha = datetime.strptime(cadena, formato)
        fechas_correctas.append(cadena)
        es_correcta = True
    except ValueError:
        try:
            # Si falla, intentar con el formato largo
            fecha = datetime.strptime(cadena, formato_largo)
            fechas_correctas.append(cadena)
            es_correcta = True
        except ValueError:
            pass # No es ninguno de los formatos esperados

    if not es_correcta:
        fechas_incorrectas.append(cadena)

print(f"Fechas correctas: {fechas_correctas}")
print(f"Fechas incorrectas: {fechas_incorrectas}")

```

Conclusión

El módulo `datetime` es una herramienta poderosa en Python. Dominar `datetime.now()`, `datetime()`, `strptime()`, `strftime()`, `timedelta` y el manejo de excepciones (`try-except ValueError`) te permitirá resolver la gran mayoría de los problemas relacionados con fechas y horas en exámenes y proyectos reales. Recuerda siempre la importancia de la coincidencia exacta de formatos al usar `strptime`.